

Hevy and Liftosaur API analysis

Overview

This report analyzes the publicly available **Hevy** and **LiftoSaur** APIs and compares them with the current implementation in the **Project-Phoenix** mobile app and **phoenix-portal** web portal. It identifies which features of these third-party APIs are currently used, documents the features that are available but unused, and proposes concrete changes to the Phoenix repositories to close the gap.

Hevy API capabilities

The Hevy API is built around **workouts**, **routines**, **routine folders**, **exercise templates**, **body-measurement data** and **workout events**. An authenticated user (via an API key) can perform CRUD operations on most of these objects. Below is a summary of the relevant endpoints and their capabilities.

Workouts

- **List workouts** – `GET /v1/workouts` returns paginated workout sessions. Each workout object contains an `id`, `title`, `start_time`, `end_time` and an array of exercises with sets. Each set includes information such as weight in kg, reps, distance, duration and RPE ¹.
- **Get a specific workout** – `GET /v1/workouts/{workoutId}` returns full details for a workout, including its exercises and sets ².
- **Create / Update workout** – `POST /v1/workouts` and `PUT /v1/workouts/{workoutId}` create or update workouts by specifying fields like `title`, `start_time`, `end_time`, notes and an array of exercise sets ³.
- **Get workout count** – `GET /v1/workouts/count` returns how many workouts exist for the user ⁴.
- **Get workout events (updates/deletions)** – `GET /v1/workouts/events` returns a paginated list of workout events with an optional `since` parameter ⁵. Clients can use this to detect changed or deleted workouts and keep their database in sync.

Routines

- **List routines** – `GET /v1/routines` returns paginated routines. A routine object includes fields such as `id`, `title`, `folder_id`, `updated_at` and nested exercises. Each exercise contains a list of sets with details like set index, type (normal, warmup, dropset, failure), weight, reps, rest, RPE or rep-range ⁶ ⁷.
- **Get a routine** – `GET /v1/routines/{id}` retrieves all exercises and sets for a specific routine.
- **Create a routine** – `POST /v1/routines` allows clients to build new routines by providing a title, folder id and an array of exercises/sets ⁸.
- **Update a routine** – `PUT /v1/routines/{id}` updates an existing routine; example payloads show updating a “Pull Day” routine’s exercises ⁹.

Routine folders

- **List routine folders** – `GET /v1/routine_folders` returns folders used to organize routines. Each folder includes `id`, `title`, `index` and timestamps ¹⁰.
- **Get routine folder** – `GET /v1/routine_folders/{id}` returns details for a particular folder ¹¹.
- **Create routine folder** – `POST /v1/routine_folders` creates a folder; the example shows creating a “Pull Day” folder ¹².

Exercise templates

- **List exercise templates** – `GET /v1/exercise_templates` returns a searchable list of built-in and user-defined exercises. Each template includes `id`, `title`, type (e.g., `weight_reps`, `duration`, `distance_duration`), primary/secondary muscle groups and a flag indicating if it is custom ¹³. Example responses show templates like bench press, squat, deadlift, running and plank ¹⁴.
- **Get a specific exercise template** – `GET /v1/exercise_templates/{id}` returns details of one template ¹⁵.

Body measurements

Although not documented in the Mintlify pages we examined, the official Swagger specification (visible on the API web page) shows endpoints for **body measurements**. These endpoints allow users to list (`GET /v1/body_measurements`), create (`POST /v1/body_measurements`) and update measurements such as body weight, waist circumference or other metrics. They are valuable for tracking progress beyond workout performance.

Authentication and subscription

Hevy uses an API key for authentication. Many endpoints require a **Hevy PRO** subscription; the API returns 401/403 status codes if the key is invalid or the user lacks PRO. The Phoenix portal code handles this by returning a `requires_pro` flag and updating integration status ¹⁶.

Liftosaur API capabilities

Liftosaur provides a scriptable system (“LiftoSCRIPT”) to define training programs. Its API exposes endpoints for **programs**, **history** (workout logs), **playground** simulation and **program statistics**.

Programs

- **List programs** – `GET /api/v1/programs` returns an array of programs, each with an `id`, a `name` and a boolean `isCurrent` indicating if it is the active program ¹⁷.
- **Get program details** – `GET /api/v1/programs/{id}` retrieves a program’s LiftoSCRIPT text ¹⁸.
- **Create a program** – `POST /api/v1/programs` accepts a request body containing a program name and LiftoSCRIPT text.
- **Update a program** – `PUT /api/v1/programs/{id}` updates an existing program’s name or script ¹⁹.
- **Delete a program** – `DELETE /api/v1/programs/{id}` deletes a program, except the current program (cannot be deleted) ¹⁹.

History (workout logs)

- **List history records** – `GET /api/v1/history` returns paginated history entries with optional `startDate`, `endDate`, `limit` and `cursor` parameters ²⁰. Each record contains an `id` and a `text` field with the Liftoscript describing the workout.
- **Get, create, update and delete history** – `GET /api/v1/history/{id}`, `POST /api/v1/history`, `PUT /api/v1/history/{id}` and `DELETE /api/v1/history/{id}` operate on individual workout records ²¹.

Playground

- **Simulate a workout** – `POST /api/v1/playground` accepts a program's Liftoscript and optional commands (e.g., `complete_set`, `change_weight`, `finish_workout`). It returns the next workout or updates the program text if the workout is completed ²². This allows clients to preview how a program will progress when sets are completed or weights changed.

Program statistics

- **Get statistics** – `POST /api/v1/program-stats` returns aggregated statistics such as number of days in the program, approximate time (minutes), number of sets and breakdown by muscle groups based on the Liftoscript ²³. This is useful for analyzing the difficulty and volume of a program before starting it.

Authentication and subscription

Liftosaur requires an **API key** (Bearer token) and a **Premium** subscription for most endpoints. The Phoenix portal passes this key in the `Authorization` header and returns a `requires_premium` flag if access is denied ²⁴.

Current usage of Hevy and Liftosaur APIs in Phoenix

Mobile application (Project-Phoenix-MP)

The Android app uses an `IntegrationManager` to connect to third-party providers. When a user clicks **connect** or **sync** for Hevy or Liftosaur, the manager calls an Edge function in the portal and expects a response containing a **list of activities** (workouts) and a status. The client maps each activity to an `ExternalActivity` object with fields for the workout name, type, start time, duration and distance. Only activities are supported – there are no fields for routines or programs ²⁵ ²⁶. The data classes `IntegrationSyncResponse` and `IntegrationActivityDto` in `IntegrationDtos.kt` likewise contain only an `activities` array and no program-level fields ²⁷.

Portal backend (phoenix-portal)

The portal implements Edge functions for synchronizing user data:

- `hevy-sync` – stores the user's API key, checks subscription and fetches workouts via `GET /v1/workouts`. It normalizes each workout into an `external_activities` row (`user_id`, `external_id`,

provider, name, started_at, duration_seconds and raw_data). It does not fetch routines, routine folders, exercise templates, body measurements or workout events ²⁸. Subscription errors set the `requires_pro` flag and update the integration status ¹⁶.

- `liftosaur-sync` – stores the Liftosaur API key, fetches workout history via `GET /api/v1/history`, and parses the Liftoscript to extract timestamp, program name, day name and duration. It constructs a workout name such as `program - dayName` or `Workout #ID` and upserts to `external_activities`. The function does **not** call `/programs`, `/playground` or `/program-stats` and therefore does not import the user's current program or its structure ²⁹ ³⁰.
- `mobile-integration-sync` – used by the mobile app. It calls the same Hevy and Liftosaur endpoints and returns a list of `activities` for mobile display, plus a `requiresUpgrade` flag for free users ³¹ ³².

Portal front-end

The portal has dedicated components for connecting to Hevy and Liftosaur:

- **HevyConnect** allows users to import workouts from a Hevy CSV file or save an API key to trigger the `hevy-sync` function. It also offers a manual sync button. The component displays subscription warnings but provides no UI for listing or importing routines or for viewing body measurements ³³ ³⁴.
- **LiftosaurConnect** accepts an API key and triggers `liftosaur-sync`. It warns that a Liftosaur Premium subscription is required. It does not show programs or program statistics ³⁵ ³⁶.
- **Integrations** page lists all connected providers and shows a table of synced activities. It has no concept of routines or programs ³⁷ ³⁸.

Gaps between available API features and current implementation

The current Phoenix implementation does **not** take advantage of many features offered by Hevy and Liftosaur:

API feature	Status in Phoenix	Evidence
Hevy routines – list, get, create, update routines (<code>GET/POST/PUT /v1/routines</code>)	Not used – neither the portal nor the mobile app requests routines from Hevy. There is no storage schema for routines or their exercises and sets, and the UI does not display routines.	The <code>hevy-sync</code> function fetches only <code>/workouts</code> ²⁸ ; <code>IntegrationManager</code> and <code>IntegrationDtos</code> support only <code>activities</code> ²⁵ .
Hevy routine folders – organizing routines into folders (<code>GET/POST /v1/routine_folders</code>)	Not used – the portal does not request or present routine folders.	No code references to <code>/routine_folders</code> or folder metadata.

API feature	Status in Phoenix	Evidence
Heavy exercise templates – list and search standard exercises (<code>GET /v1/exercise_templates</code>)	Not used – Phoenix uses only custom exercises saved in its own database; there is no call to <code>exercise_templates</code> to map Heavy exercise IDs to Phoenix equivalents.	
Heavy workout events – detect updates or deletions (<code>GET /v1/workouts/events</code>)	Not used – the sync functions only perform full imports and do not check for modifications since the last sync. As a result, updates or deletions in Heavy are not reflected in Phoenix.	
Heavy body measurements – log weight or body composition data	Not used – there is no integration for body measurements.	
LiftoSaur programs – list, create, update and delete programs (<code>GET/POST/PUT/DELETE /api/v1/programs</code>)	Not used – <code>liftoSaur-sync</code> only imports workout history. Phoenix cannot display the user's current training program or program library.	The sync function calls <code>/history</code> only ²⁹ .
LiftoSaur playground – simulate program progression with commands (<code>POST /api/v1/playground</code>)	Not used – there is no support for previewing upcoming workouts or updating program text when sets are completed.	
LiftoSaur program statistics – compute program workload (<code>POST /api/v1/program-stats</code>)	Not used – there is no call to this endpoint; Phoenix cannot show users statistics about their programs (volume, time per session, muscle group distribution).	
Sync gating consistency – unify subscription checks between mobile and portal	The mobile <code>mobile-integration-sync</code> returns a <code>requiresUpgrade</code> flag but the portal uses a different gating (<code>requires_pro / requires_premium</code>) and stores errors in <code>user_integrations</code> . This leads to inconsistent user experience and error messages.	

Proposed changes to Phoenix repositories

To close the gap between what the APIs offer and what Phoenix currently provides, several changes are required in both the **portal** and the **mobile app**. The proposals below focus on architecture and specific files requiring modifications.

1. Extend the data model to support programs and routines

- **Create new tables** in Supabase: `external_programs` and `external_routines`.
`external_programs` should store `user_id`, `provider` (hevy or liftosaur), `external_id`, `name`, `is_current` (where applicable), `raw_data` (JSON or text) and timestamps.
`external_routines` should store routine metadata and include a relationship to program or folder where appropriate.
- **Modify** `IntegrationActivityDto` and `IntegrationSyncResponse` (in **Project-Phoenix-MP**) to include optional arrays `programs` and `routines`. Similarly, update the `ExternalActivity` data class to handle programs and routines.
- **Update** `IntegrationManager` to map the new fields from the sync response and store them in local persistence (e.g., database). Provide UI cues in the mobile app to display the list of programs/ routines imported from Hevy or Liftosaur.

2. Import routines from Hevy

- **Portal backend** – update `hevy-sync` and `mobile-integration-sync` functions to call `GET /v1/routines` after fetching workouts. For each routine, upsert into `external_routines` and link to the user. Use pagination to fetch all routines (the API accepts `page` and `pageSize` parameters ⁶). If the user has routine folders, call `GET /v1/routine_folders` and store folder names so routines can be grouped ¹⁰.
- **Sync incremental updates** – call `GET /v1/workouts/events` with the `since` timestamp stored in `last_sync_at` to process updated or deleted workouts ⁵. This prevents duplicating existing workouts and allows Phoenix to remove or update workouts that changed in Hevy.

3. Use Hevy exercise templates for mapping exercises

- **Fetch templates** – add a function to call `GET /v1/exercise_templates` and cache the list of exercises (bench press, squat, deadlift, etc.) with their `id`, name and muscle groups ¹³. Use this mapping when importing workouts or routines: if the Hevy workout references an exercise ID, look up the template to populate the exercise name and targeted muscles. This improves display quality in Phoenix and allows future analytics by muscle group.

4. Import body-measurement data

- **Portal backend** – implement a new Edge function (e.g., `hevy-measurements-sync`) that calls `GET /v1/body_measurements` and stores the data in a `body_measurements` table. On the mobile and portal UI, provide a “Measurements” section where users can see their weight, body fat and other metrics over time.

5. Import programs and statistics from Liftosaur

- **Portal backend** – extend `liftosaur-sync` to call `GET /api/v1/programs` to fetch the list of programs ¹⁷. For each program, call `GET /api/v1/programs/{id}` to retrieve the Liftoscript text ¹⁸ and store it in `external_programs`. If the program has `isCurrent = true`, mark it as the active program.
- **Fetch program stats** – call `POST /api/v1/program-stats` with the program text to compute days, approximate minutes, set counts and muscle group breakdown ²³. Store these statistics in a `program_stats` table and display them in the UI to help users understand program load.

6. Support Liftosaur playground simulation

- **Add a new endpoint** – create an Edge function (e.g., `liftosaur-playground`) that forwards requests to `POST /api/v1/playground`. Allow the mobile or portal UI to send commands (complete set, change weight, finish workout) and show the predicted next workout. When the user finishes a workout, update the program text in `external_programs` accordingly ²².

7. Enhance UI and user experience

- **Update portal UI** – modify `HevyConnect` and `LiftosaurConnect` components to show the number of imported routines/programs and provide buttons to view them. Add new sections in the **Integrations** page where users can browse their imported routines and programs. For Liftosaur, include a simulation button that opens the playground.
- **Mobile UI** – update screens to display programs and routines alongside workouts. Provide progress indicators for the user's current program (e.g., which day of the program they are on). Use `program_stats` to show users how much time and volume remain in the program.
- **Consistent subscription messaging** – unify the subscription gating flags (`requires_pro`, `requires_premium`, `requiresUpgrade`) to a single boolean field (e.g., `requiresUpgrade`) and ensure both the portal and mobile app check and display the same message. This will reduce confusion and make gating logic easier to maintain.

8. Extend API error handling and resilience

- **Retry and partial sync** – implement retry logic and limit loops when fetching pages. Add support for processing partial pages of routines or programs to avoid timeouts.
- **Conflict resolution** – define how updates and deletions from Hevy or Liftosaur should propagate to Phoenix. Use the workout events endpoint and program IDs to synchronize deletes and updates.

Conclusion

Both Hevy and Liftosaur offer rich APIs that allow users to manage workout routines, programs, exercise templates, body measurements, program statistics and more. The current Phoenix implementation only imports basic workout history. By extending the data model, updating sync functions to fetch routines/programs/statistics, leveraging exercise templates and body measurements, and enhancing the UI, Phoenix can provide users with a more complete view of their training data. The proposed changes will allow Phoenix to match the capabilities of these third-party APIs, improve user engagement and reduce gaps between the mobile and web experiences.

1 3 4 **mintlify.wiki**

<https://mintlify.wiki/imalfect/hevy-http-mcp/api-reference/workouts.md>

2 5 **hevy-mcp/openapi-spec.json at main · chrisdoc/hevy-mcp · GitHub**

<https://github.com/chrisdoc/hevy-mcp/blob/main/openapi-spec.json>

6 7 8 9 **mintlify.wiki**

<https://mintlify.wiki/imalfect/hevy-http-mcp/api-reference/routines.md>

10 11 12 **mintlify.wiki**

<https://mintlify.wiki/imalfect/hevy-http-mcp/api-reference/routine-folders.md>

13 14 15 **mintlify.wiki**

<https://mintlify.wiki/imalfect/hevy-http-mcp/api-reference/exercise-templates.md>

16 28 **index.ts**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/HEAD/supabase/functions/hevy-sync/index.ts>

17 18 19 20 21 22 23 **REST API - Liftosaur Documentation**

<https://www.liftosaur.com/doc/api>

24 29 30 **index.ts**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/HEAD/supabase/functions/liftosaur-sync/index.ts>

25 26 **IntegrationManager.kt**

<https://github.com/9thLevelSoftware/Project-Phoenix-MP/blob/ff27731e3ed4c30bbb51ff3bf021a04b6192bc68/shared/src/commonMain/kotlin/com/devil/phoenixproject/data/integration/IntegrationManager.kt>

27 **IntegrationDtos.kt**

<https://github.com/9thLevelSoftware/Project-Phoenix-MP/blob/HEAD/shared/src/commonMain/kotlin/com/devil/phoenixproject/data/sync/IntegrationDtos.kt>

31 32 **index.ts**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/bc40ad4f3e3fa5de8a59f3c08f73afbe5f1bf315/supabase/functions/mobile-integration-sync/index.ts>

33 34 **HevyConnect.tsx**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/HEAD/src/app/components/integrations/HevyConnect.tsx>

35 36 **LiftosaurConnect.tsx**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/HEAD/src/app/components/integrations/LiftosaurConnect.tsx>

37 38 **Integrations.tsx**

<https://github.com/9thLevelSoftware/phoenix-portal/blob/HEAD/src/app/components/Integrations.tsx>